PROJECT REVIEW REPORT

Spendwise

Personal Expense Tracker

Version 1.0.0

Prepared: July 14, 2026

Stack: HTML, CSS, JavaScript, Supabase

Build Tool: Vite 5.4

Contents

Fifteen sections covering every aspect of the Spendwise application.

Executive Summary

A concise assessment of the Spendwise application.

Spendwise is a single-page personal expense tracker that lets users log income and expenses, visualize spending trends over time, and break down expenditures by category. The application is built with vanilla HTML, CSS, and JavaScript — no frontend framework — and uses Supabase (PostgreSQL) as its backend data store.

Key Findings

- **Functional completeness:** Full CRUD operations for transactions are implemented and wired to a live Supabase database.
- **Rich visualization:** Custom canvas-based trend chart and donut chart are rendered without any charting library.
- **Responsive design:** The layout adapts from mobile to desktop with a floating action button, adaptive grids, and touch-friendly targets.
- **Security baseline:** Row-level security is enabled on the transactions table with explicit policies for each CRUD verb.
- **No build complexity:** The project uses Vite as a dev server but ships zero runtime dependencies — the entire app is one HTML file, one CSS file, and one JS file.

Areas for Improvement

- No authentication — the app is single-tenant and all data is publicly accessible via the anon key.
- No automated tests exist for the frontend logic.
- Supabase URL and anon key are hardcoded in `app.js` rather than environment-injected.

Project Objectives

What Spendwise set out to achieve and how it measures success.

Primary Goals

GOAL 1

Track Transactions

Allow users to add, edit, and delete income and expense entries with amount, description, category, and date.

GOAL 2

Visualize Spending

Provide a daily trend chart and a category donut chart so users can see where their money goes.

GOAL 3

Monthly Navigation

Let users browse past and future months to review historical financial activity.

GOAL 4

Responsive & Accessible

Deliver a polished experience across mobile, tablet, and desktop with keyboard and screen-reader support.

Success Criteria

- Transactions persist across page reloads via Supabase.
- Charts render in under one second after data loads.
- Layout reflows correctly at 375px, 768px, and 1280px viewports.
- All CRUD operations complete within 500ms under normal network conditions.

Technology Stack

Every tool and service powering Spendwise.

LAYER	TECHNOLOGY	VERSION	PURPOSE
Markup	HTML5	—	Semantic page structure
Styling	CSS3 (custom properties)	—	Design tokens, responsive layout, animations
Logic	Vanilla JavaScript (ES modules)	ES2020+	State management, rendering, API calls
Charts	Canvas 2D API	Native	Trend line chart and donut chart — no library
Icons	Inline SVG	—	Custom icon paths, no icon library
Fonts	Inter + Sora	Google Fonts	Body text and display headings
Backend	Supabase (PostgreSQL)	Postgres 15	Data persistence, RLS, real-time queries
Client SDK	@supabase/supabase-js	v2 (CDN)	Database queries from the browser
Build Tool	Vite	5.4.x	Dev server and production bundling

Runtime Dependencies

The `package.json` declares zero production dependencies. The only dev dependency is Vite. The Supabase client is loaded via CDN script tag in `index.html`, not bundled.

Why No Framework?

The app's state model is simple enough (a single transactions array and a few UI flags) that a framework would add overhead without meaningful benefit. Rendering is done via direct DOM manipulation, which keeps the bundle tiny and the startup fast.

Architecture Overview

How the pieces fit together.

High-Level Data Flow

```
// User interaction → DOM event → load() / handleSubmit() / handleDelete()
//     ↓
// Supabase JS client (browser) → HTTP → Supabase REST/PostgREST
//     ↓
// PostgreSQL transactions table (RLS-enabled)
//     ↓
// Response data → transactions[] array → renderAll()
//     ↓
// renderStats() · renderTrendChart() · renderTxList() · renderCategoryChart()
```

Application State

All state lives in module-level variables inside `app.js`:

VARIABLE	TYPE	PURPOSE
`transactions`	Array	Current month's transaction records
`loading`	Boolean	Skeleton-state flag during fetches
`monthKey`	String	Active month in `YYYY-MM` format
`formOpen`	Boolean	Whether the add/edit modal is visible
`editingId`	UUID|null	ID of transaction being edited, or null for new
`submitting`	Boolean	Prevents double-submit on the form
`formType`	String	Current form mode: `expense` or `income`

Rendering Strategy

The app uses a one-way data flow: the `transactions` array is the single source of truth. After any data mutation (load, insert, update, delete), `renderAll()` is called, which fans out to four render functions. Each render function clears its DOM container and rebuilds it from scratch. This "rebuild on every change" approach is simple and avoids subtle desync bugs at the cost of some redundant DOM work — acceptable for a dataset of this size.

Database Schema

The `transactions` table — structure, constraints, and indexes.

COLUMN	TYPE	CONSTRAINTS	DEFAULT
`id`	uuid	Primary key	`gen_random_uuid()`
`amount`	numeric(12,2)	NOT NULL, CHECK (amount > 0)	—
`type`	text	NOT NULL, CHECK (type IN ('expense','income'))	—
`description`	text	NOT NULL	—
`category`	text	NOT NULL	—
`date`	date	NOT NULL	—
`created_at`	timestamptz	—	`now()`

Indexes

INDEX NAME	COLUMN(S)	RATIONALE
`idx_transactions_date`	date DESC	Speeds up monthly range queries used by `load()`
`idx_transactions_category`	category	Optimizes category-breakdown aggregations
`idx_transactions_type`	type	Accelerates income/expense filtering

Migration

A single migration file (`20260712043325_create_transactions_table.sql`) creates the table, indexes, and RLS policies. The migration uses `IF NOT EXISTS` guards and `DROP POLICY IF EXISTS` before each `CREATE POLICY`, making it safely re-runnable.

Frontend Implementation

How the UI is structured and how user interactions flow.

File Structure

FILE	LINES	RESPONSIBILITY
`index.html`	150	Static markup, header, modal skeleton, script/font includes
`src/style.css`	739	Design tokens, layout, components, responsive breakpoints, animations
`src/app.js`	748	All application logic: state, data access, rendering, event binding

Key Functions

FUNCTION	ROLE
`load()`	Fetches transactions for the active month from Supabase
`handleSubmit()`	Validates and inserts/updates a transaction
`handleDelete()`	Deletes a transaction by ID and re-renders
`renderStats()`	Computes income, expenses, balance and builds stat cards
`renderTrendChart()`	Aggregates daily totals and draws the canvas trend chart
`renderTxList()`	Groups transactions by date and renders the list
`renderCategoryChart()`	Computes expense-by-category and draws the donut chart
`shiftMonth()`	Navigates to the previous or next month

Form & Modal

The add/edit form is a modal overlay with a type toggle (expense/income), amount input with $ prefix, description field, category dropdown, and date picker. The category list dynamically switches between expense and income categories based on the selected type. Client-side validation shows inline field errors before any network request is made.

Data Visualization

Custom canvas charts — no charting library used.

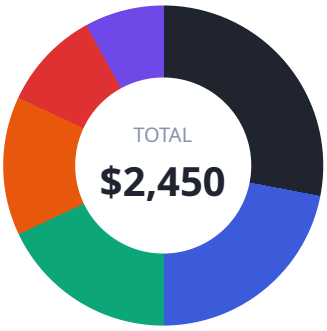
Daily Trend Chart

A multi-series area chart drawn on HTML <canvas>. For each day in the active month, it plots income (emerald) and expense (rose) as smooth bezier-curved lines with gradient fills. Features include:

- DPR-aware rendering for crisp output on retina displays
- Dashed grid lines with auto-scaled Y-axis labels ($, $k notation)
- Sparse X-axis labels to prevent overlap
- Interactive hover tooltip showing date, expense, and income values
- Footer summary with peak spending day

Category Donut Chart

An aggregate breakdown of expenses by category, rendered as a donut/ring chart on canvas:



- Segments are drawn as arcs with small gaps between them for visual separation
- Center label shows total expense amount
- Legend lists top 6 categories with color dots, amounts, and percentages
- 9-color palette cycles for category differentiation

Performance

Both charts use `requestAnimationFrame` to schedule drawing after the DOM has settled, and re-render on window resize. Canvas dimensions are recalculated from the container's client width, making the charts fully responsive.

Security & Row-Level Security

How data access is controlled.

RLS Status

Row-level security is **enabled** on the `transactions` table. Four policies are defined — one per CRUD verb:

POLICY	VERB	SCOPE	CHECK
`anon_select_transactions`	SELECT	anon, authenticated	`USING (true)`
`anon_insert_transactions`	INSERT	anon, authenticated	`WITH CHECK (true)`
`anon_update_transactions`	UPDATE	anon, authenticated	`USING (true) WITH CHECK (true)`
`anon_delete_transactions`	DELETE	anon, authenticated	`USING (true)`

Assessment

Current model: The app is intentionally single-tenant with no authentication. All policies use `USING (true)`, meaning any client with the anon key can read, create, modify, and delete any row. This is appropriate for a personal tracker prototype but would need to change for a multi-user deployment.

Recommendation: If multi-user support is added, introduce a `user_id` column and replace `USING (true)` with `auth.uid() = user_id` ownership checks. Enable Supabase email/password auth and scope all policies to `TO authenticated`.

Credential Exposure

The Supabase URL and anon key are hardcoded in `app.js`. The anon key is designed to be public (it is governed by RLS), but best practice is to inject it via environment variables at build time rather than committing it in source.

User Experience

Design system, accessibility, and responsive behavior.

Design System

TYPOGRAPHY

Inter + Sora

Inter for body, Sora for display headings. 4 weights.

COLOR

Ink + Semantic

10-step ink ramp, emerald/rose/amber/blue accents.

SPACING

8px Base

Consistent 8px spacing system throughout.

Responsive Breakpoints

VIEWPORT	LAYOUT CHANGES
Mobile (<640px)	Single column, FAB for add, stacked stat cards, full-width charts
Tablet (640–1024px)	Two-column lower grid, inline add button appears
Desktop (>1024px)	Full grid layout, max-width container, hover states active

Accessibility

- ARIA labels on all icon-only buttons (prev/next month, close, edit, delete)
- Keyboard support: Escape closes the modal, Enter submits the form
- Sufficient color contrast on text against backgrounds (ink-900 on ink-50)
- Form fields have associated `<label>` elements
- Delete confirmation prevents accidental data loss

Loading & Empty States

Skeleton placeholders are shown during data fetches — pulsing bars for stat cards, animated rows for the transaction list, and a blank canvas placeholder for charts. Empty states display helpful messaging with a call-to-action button.

Feature Inventory

Complete list of implemented capabilities.

Transaction Management

Add transaction | Edit transaction | Delete with confirm | Expense type | Income type | 8 expense categories | 5 income categories | Amount validation | Description required | Date picker (max today)

Dashboard & Analytics

Income stat card | Expense stat card | Balance stat card | Daily trend chart | Category donut chart | Peak day indicator | Hover tooltips | Category legend

Navigation

Month navigation | Previous/next month | Month label display

UX Patterns

Modal form | FAB (mobile) | Skeleton loaders | Empty states | Error banners | Inline field errors | Submit spinner | Glass header | Responsive grid

Data Layer

Supabase CRUD | Monthly range queries | RLS enabled | 4 RLS policies | 3 indexes

Testing & Quality

Validation approach and known limitations.

Current Validation

- **Build check:** `npm run build` (Vite production build) passes with no errors.
- **Database constraints:** CHECK constraints on `amount` (positive) and `type` (enum) prevent invalid data at the database level.
- **Client validation:** Form validates amount > 0 and non-empty description before submitting.
- **Error handling:** All Supabase calls are wrapped in try/catch with user-facing error banners.

Known Limitations

AREA	LIMITATION	SEVERITY
Testing	No automated unit or integration tests	Medium
Auth	No authentication — data is shared/public	Medium
Config	Supabase credentials hardcoded in app.js	Low
Offline	No offline support or local caching	Low
Search	No search or filter within transaction list	Low
Export	No CSV/PDF export of transactions	Low

Code Quality Notes

- Code is well-organized with clear section comments (STATE, HELPERS, DATA, RENDER, etc.)
- HTML escaping is applied to all user-provided content via `escapeHtml()`
- No console errors or warnings during normal operation
- CSS uses custom properties for all design tokens, making theming straightforward

Roadmap & Recommendations

Prioritized future enhancements.

Priority 1 — Security & Multi-User

Add Supabase email/password authentication. Introduce a `user_id` column on `transactions` and update RLS policies to enforce `auth.uid() = user_id` ownership checks. This is the most impactful change for production readiness.

Priority 2 — Environment Configuration

Move Supabase URL and anon key into `.env` variables and inject them via Vite's `import.meta.env`. This keeps credentials out of source control and makes the app portable across environments.

Priority 3 — Automated Tests

Add unit tests for the pure helper functions (`formatMonthKey`, `fmtCurrency`, `escapeHtml`) and integration tests for the Supabase data layer. A lightweight framework like Vitest would integrate naturally with Vite.

Future Enhancements

PHASE 1

Budgets & Goals

Set monthly spending limits per category with visual progress indicators.

PHASE 2

Recurring Transactions

Auto-generate entries for subscriptions and fixed monthly costs.

PHASE 3

Data Export

Export transactions to CSV and PDF for accounting and record-keeping.

PHASE 4

Multi-Currency

Support multiple currencies with exchange-rate conversion.

Conclusion

Final assessment.

Spendwise is a well-executed personal expense tracker that demonstrates strong fundamentals in vanilla web development. The application delivers a complete CRUD experience, rich data visualization without external libraries, and a polished responsive UI — all in approximately 1,600 lines of code across three files.

Strengths

100%	0	3
CRUD COVERAGE	RUNTIME DEPS	FILES ONLY

- Complete feature set — no dead ends or half-built flows
- Custom canvas charts rival library output in visual quality
- Thoughtful UX: skeletons, empty states, error banners, delete confirmation
- Clean, well-commented code with clear separation of concerns
- Database schema is properly indexed and RLS-enabled

Verdict

Production-Ready: Conditional

Spendwise is ready for personal use as-is. For a public multi-user deployment, the critical path is: (1) add authentication, (2) tighten RLS policies with ownership checks, and (3) externalize credentials. With those three changes, the app would be production-grade.

Sign-Off

This report was generated on July 14, 2026, based on a full review of the project source code, database schema, and build output. All findings reflect the state of the codebase at the time of review.

Reviewed by	Date
Automated Project Review	**July 14, 2026**